

LOGICAL STRUCTURES

Lecture 3. Normal forms

Advanced Program in Computer Science

Spring, 2025

Contents

- ➊ Semantic equivalence, satisfiability and validity
- ➋ Conjunctive normal forms and validity
- ➌ Horn clauses and satisfiability
- ➍ SAT solvers

1. Semantic equivalence, satisfiability and validity

1. Semantic equivalence, satisfiability and validity

Two formulas ϕ and ψ are said to be equivalent if they have the same ‘meaning’.

1. Semantic equivalence, satisfiability and validity

Two formulas ϕ and ψ are said to be equivalent if they have the same ‘meaning’.

Definition 2.1

Let ϕ and ψ be formulas of propositional logic. We say that ϕ and ψ are *semantically equivalent* iff $\phi \models \psi$ and $\psi \models \phi$ hold. In that case, we write $\phi \equiv \psi$.

1. Semantic equivalence, satisfiability and validity

Two formulas ϕ and ψ are said to be equivalent if they have the same ‘meaning’.

Definition 2.1

Let ϕ and ψ be formulas of propositional logic. We say that ϕ and ψ are *semantically equivalent* iff $\phi \models \psi$ and $\psi \models \phi$ hold. In that case, we write $\phi \equiv \psi$. Further, we call ϕ *valid* if $\models \phi$ holds.

1. Semantic equivalence, satisfiability and validity

Two formulas ϕ and ψ are said to be equivalent if they have the same ‘meaning’.

Definition 2.1

Let ϕ and ψ be formulas of propositional logic. We say that ϕ and ψ are *semantically equivalent* iff $\phi \models \psi$ and $\psi \models \phi$ hold. In that case, we write $\phi \equiv \psi$. Further, we call ϕ *valid* if $\models \phi$ holds.

Note: An equivalent definition for $\phi \equiv \psi$ is $\models (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi)$.

Example

Examples of equivalent formulas

$$p \rightarrow q \equiv \neg q \rightarrow \neg p$$

$$p \rightarrow q \equiv \neg p \vee q$$

$$p \wedge q \rightarrow p \equiv r \vee \neg r$$

$$p \wedge q \rightarrow r \equiv p \rightarrow (q \rightarrow r).$$

Tautology and Validity

Recall: ϕ is a tautology iff $\models \phi$ holds.

Tautology and Validity

Recall: ϕ is a tautology iff $\models \phi$ holds.

Lemma 2.2

*Given formulas $\phi_1, \phi_2, \dots, \phi_n$ and ψ of propositional logic,
 $\phi_1, \phi_2, \dots, \phi_n \models \psi$ holds iff $\models (\phi_1 \rightarrow (\phi_2 \rightarrow (\phi_3 \rightarrow \dots \rightarrow (\phi_n \rightarrow \psi))))$
holds.*

A strategy

We want to transform formulas into ones which don't contain $\rightarrow$ and the occurrence of $\wedge$ and $\vee$ are confined to separate layers such that validity checks are easy.

A strategy

We want to transform formulas into ones which don't contain $\rightarrow$ and the occurrence of $\wedge$ and $\vee$ are confined to separate layers such that validity checks are easy.

Strategy:

- using the equivalent $\phi \rightarrow \psi \equiv \neg\phi \vee \psi$ to remove all occurrence of $\rightarrow$;

A strategy

We want to transform formulas into ones which don't contain $\rightarrow$ and the occurrence of $\wedge$ and $\vee$ are confined to separate layers such that validity checks are easy.

Strategy:

- using the equivalent $\phi \rightarrow \psi \equiv \neg\phi \vee \psi$ to remove all occurrence of $\rightarrow$;
- transform a formula without any $\rightarrow$ into *normal form* (easy to check for validity).

Conjunctive Normal Form (CNF)

Definition 2.3

A *literal* L is either an atom p or the negation of an atom $\neg p$.

Conjunctive Normal Form (CNF)

Definition 2.3

A **literal** L is either an atom p or the negation of an atom $\neg p$. A formula C is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where clause D is a disjunction of literals:

$$L :: p \mid \neg p$$

$$D :: L \mid L \vee D$$

$$C :: D \mid D \wedge C$$

Conjunctive Normal Form (CNF)

Definition 2.3

A **literal** L is either an atom p or the negation of an atom $\neg p$. A formula C is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where clause D is a disjunction of literals:

$$L :: p | \neg p$$

$$D :: L | L \vee D$$

$$C :: D | D \wedge C$$

Example 2.4

$$(i) (\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$$

Conjunctive Normal Form (CNF)

Definition 2.3

A **literal** L is either an atom p or the negation of an atom $\neg p$. A formula C is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where clause D is a disjunction of literals:

$$L :: p | \neg p$$

$$D :: L | L \vee D$$

$$C :: D | D \wedge C$$

Example 2.4

(i) $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ is in CNF

Conjunctive Normal Form (CNF)

Definition 2.3

A **literal** L is either an atom p or the negation of an atom $\neg p$. A formula C is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where clause D is a disjunction of literals:

$$L :: p | \neg p$$

$$D :: L | L \vee D$$

$$C :: D | D \wedge C$$

Example 2.4

- (i) $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ is in CNF
- (ii) $(p \vee r) \wedge (\neg p \vee r) \wedge (p \vee \neg r)$

Conjunctive Normal Form (CNF)

Definition 2.3

A **literal** L is either an atom p or the negation of an atom $\neg p$. A formula C is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where clause D is a disjunction of literals:

$$L :: p | \neg p$$

$$D :: L | L \vee D$$

$$C :: D | D \wedge C$$

Example 2.4

- (i) $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ is in CNF
- (ii) $(p \vee r) \wedge (\neg p \vee r) \wedge (p \vee \neg r)$ is in CNF

Conjunctive Normal Form (CNF)

Definition 2.3

A **literal** L is either an atom p or the negation of an atom $\neg p$. A formula C is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where clause D is a disjunction of literals:

$$L :: p | \neg p$$

$$D :: L | L \vee D$$

$$C :: D | D \wedge C$$

Example 2.4

- (i) $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ is in CNF
- (ii) $(p \vee r) \wedge (\neg p \vee r) \wedge (p \vee \neg r)$ is in CNF
- (iii) $(\neg(q \vee p) \vee r) \wedge (q \vee r)$

Conjunctive Normal Form (CNF)

Definition 2.3

A **literal** L is either an atom p or the negation of an atom $\neg p$. A formula C is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where clause D is a disjunction of literals:

$$L :: p | \neg p$$

$$D :: L | L \vee D$$

$$C :: D | D \wedge C$$

Example 2.4

- (i) $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ is in CNF
- (ii) $(p \vee r) \wedge (\neg p \vee r) \wedge (p \vee \neg r)$ is in CNF
- (iii) $(\neg(q \vee p) \vee r) \wedge (q \vee r)$ is **NOT** in CNF

Conjunctive Normal Form (CNF)

Definition 2.3

A **literal** L is either an atom p or the negation of an atom $\neg p$. A formula C is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where clause D is a disjunction of literals:

$$L :: p | \neg p$$

$$D :: L | L \vee D$$

$$C :: D | D \wedge C$$

Example 2.4

- (i) $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ is in CNF
- (ii) $(p \vee r) \wedge (\neg p \vee r) \wedge (p \vee \neg r)$ is in CNF
- (iii) $(\neg(q \vee p) \vee r) \wedge (q \vee r)$ is **NOT** in CNF since $q \vee p$ is not a literal.

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Answer: They allow easy checks of validity.

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Answer: They allow easy checks of validity.

Suppose that we consider the formula $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$.

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Answer: They allow easy checks of validity.

Suppose that we consider the formula $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$. The semantic entailment $\models (\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Answer: They allow easy checks of validity.

Suppose that we consider the formula $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$. The semantic entailment $\models (\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ holds iff

$$\models \neg q \vee p \vee r \qquad \models \neg p \vee r \qquad \models q$$

hold separately.

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Answer: They allow easy checks of validity.

Suppose that we consider the formula $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$. The semantic entailment $\models (\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ holds iff

$$\models \neg q \vee p \vee r \qquad \models \neg p \vee r \qquad \models q$$

hold separately.

Lemma 2.5

A disjunction of literals $L_1 \vee L_2 \vee \dots \vee L_m$ is valid iff there are $1 \leq i, j \leq m$ such that L_i is $\neg L_j$.

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Answer: They allow easy checks of validity.

Suppose that we consider the formula $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$. The semantic entailment $\models (\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ holds iff

$$\models \neg q \vee p \vee r \qquad \models \neg p \vee r \qquad \models q$$

hold separately.

Lemma 2.5

A disjunction of literals $L_1 \vee L_2 \vee \dots \vee L_m$ is valid iff there are $1 \leq i, j \leq m$ such that L_i is $\neg L_j$.

Example 2.6

$$\textcircled{1} \quad p \vee q \vee r \vee \neg q$$

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Answer: They allow easy checks of validity.

Suppose that we consider the formula $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$. The semantic entailment $\models (\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ holds iff

$$\models \neg q \vee p \vee r \qquad \models \neg p \vee r \qquad \models q$$

hold separately.

Lemma 2.5

A disjunction of literals $L_1 \vee L_2 \vee \dots \vee L_m$ is valid iff there are $1 \leq i, j \leq m$ such that L_i is $\neg L_j$.

Example 2.6

- ① $p \vee q \vee r \vee \neg q$ is valid.

Conjunctive Normal Form (CNF)

Question: Why do we need formulas in CNF?

Answer: They allow easy checks of validity.

Suppose that we consider the formula $(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$. The semantic entailment $\models (\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$ holds iff

$$\models \neg q \vee p \vee r \qquad \models \neg p \vee r \qquad \models q$$

hold separately.

Lemma 2.5

A disjunction of literals $L_1 \vee L_2 \vee \dots \vee L_m$ is valid iff there are $1 \leq i, j \leq m$ such that L_i is $\neg L_j$.

Example 2.6

- ① $p \vee q \vee r \vee \neg q$ is valid.
- ② $\neg q \vee p \vee r$ is not valid.

Satisfiability

Before we study how to compute equivalent conjunctive normal forms, we introduce another semantic concept closely related to that of validity

Satisfiability

Before we study how to compute equivalent conjunctive normal forms, we introduce another semantic concept closely related to that of validity

Definition 2.7

Given a formula ϕ in propositional logic, we say that ϕ is *satisfiable* if it has a valuation in which it evaluates to **T**.

Satisfiability

Before we study how to compute equivalent conjunctive normal forms, we introduce another semantic concept closely related to that of validity

Definition 2.7

Given a formula ϕ in propositional logic, we say that ϕ is *satisfiable* if it has a valuation in which it evaluates to **T**.

Example 2.8

$p \vee q \rightarrow p$ is satisfiable since it computes T if we assign T to p .

Satisfiability

Before we study how to compute equivalent conjunctive normal forms, we introduce another semantic concept closely related to that of validity

Definition 2.7

Given a formula ϕ in propositional logic, we say that ϕ is *satisfiable* if it has a valuation in which it evaluates to **T**.

Example 2.8

$p \vee q \rightarrow p$ is satisfiable since it computes T if we assign T to p .
Note that $p \vee q \rightarrow p$ is not valid.

Satisfiability

Before we study how to compute equivalent conjunctive normal forms, we introduce another semantic concept closely related to that of validity

Definition 2.7

Given a formula ϕ in propositional logic, we say that ϕ is *satisfiable* if it has a valuation in which it evaluates to **T**.

Example 2.8

$p \vee q \rightarrow p$ is satisfiable since it computes T if we assign T to p .
Note that $p \vee q \rightarrow p$ is not valid.

Proposition 2.1

Let ϕ be a formula of propositional logic. Then ϕ is satisfiable iff $\neg\phi$ is not valid.

CNF from truth-table

Consider a formula ϕ which has the following truth table

p	q	r	ϕ
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	T

CNF from truth-table

Consider a formula ϕ which has the following truth table

p	q	r	ϕ
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	T

Look at entries which compute **F**:

CNF from truth-table

Consider a formula ϕ which has the following truth table

p	q	r	ϕ
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	T

Look at entries which compute **F**:

Line 2: $\psi_1 = \neg p \vee \neg q \vee r$

CNF from truth-table

Consider a formula ϕ which has the following truth table

p	q	r	ϕ
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	T

Look at entries which compute **F**:

Line 2: $\psi_1 = \neg p \vee \neg q \vee r$

Line 5: $\psi_2 = p \vee \neg q \vee \neg r$

CNF from truth-table

Consider a formula ϕ which has the following truth table

p	q	r	ϕ
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	T

Look at entries which compute **F**:

Line 2: $\psi_1 = \neg p \vee \neg q \vee r$

Line 5: $\psi_2 = p \vee \neg q \vee \neg r$

Line 6: $\psi_3 = p \vee \neg q \vee r$

CNF from truth-table

Consider a formula ϕ which has the following truth table

p	q	r	ϕ
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	T

Look at entries which compute **F**:

Line 2: $\psi_1 = \neg p \vee \neg q \vee r$

Line 5: $\psi_2 = p \vee \neg q \vee \neg r$

Line 6: $\psi_3 = p \vee \neg q \vee r$

Line 7: $\psi_4 = p \vee q \vee \neg r$

CNF from truth-table

Consider a formula ϕ which has the following truth table

p	q	r	ϕ
T	T	T	T
T	T	F	F
T	F	T	T
T	F	F	T
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	T

Look at entries which compute **F**:

Line 2: $\psi_1 = \neg p \vee \neg q \vee r$

Line 5: $\psi_2 = p \vee \neg q \vee \neg r$

Line 6: $\psi_3 = p \vee \neg q \vee r$

Line 7: $\psi_4 = p \vee q \vee \neg r$

Then a CNF of ϕ is

$\psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4$

It should be clear by now that a full truth table of ϕ and an equivalent formula in CNF are pretty much the same thing as far as questions about validity are concerned – although the formula in CNF may be much more compact.

Example

Construct a formula in CNF based on each of the following truth tables

p	q	ϕ_1
T	T	F
F	T	F
T	F	F
F	F	T

r	s	q	ϕ_3
T	T	T	F
T	T	F	T
T	F	T	T
F	T	T	F
T	F	F	T
F	T	F	T
F	F	T	F
F	F	F	T

2. Conjunctive normal forms and validity

CNF algorithm

2. Conjunctive normal forms and validity

CNF algorithm

Algorithm CNF:

- (1) CNF terminates for all formulas of propositional logic as input;
- (2) for each such input, CNF outputs an equivalent formula; and
- (3) all output computed by CNF is in CNF.

2. Conjunctive normal forms and validity

CNF algorithm

Algorithm CNF:

- (1) CNF terminates for all formulas of propositional logic as input;
- (2) for each such input, CNF outputs an equivalent formula; and
- (3) all output computed by CNF is in CNF.

Preprocessing: Input ϕ

2. Conjunctive normal forms and validity

CNF algorithm

Algorithm CNF:

- (1) CNF terminates for all formulas of propositional logic as input;
- (2) for each such input, CNF outputs an equivalent formula; and
- (3) all output computed by CNF is in CNF.

Preprocessing: Input ϕ

- replace all subformulas of the form $\psi \rightarrow \eta$ by $\neg\psi \vee \eta$ using a procedure called IMPL_FREE;

2. Conjunctive normal forms and validity

CNF algorithm

Algorithm CNF:

- (1) CNF terminates for all formulas of propositional logic as input;
- (2) for each such input, CNF outputs an equivalent formula; and
- (3) all output computed by CNF is in CNF.

Preprocessing: Input ϕ

- replace all subformulas of the form $\psi \rightarrow \eta$ by $\neg\psi \vee \eta$ using a procedure called `IMPL_FREE`;
- formulas of the form $\neg(p \wedge q)$ may appear,

2. Conjunctive normal forms and validity

CNF algorithm

Algorithm CNF:

- (1) CNF terminates for all formulas of propositional logic as input;
- (2) for each such input, CNF outputs an equivalent formula; and
- (3) all output computed by CNF is in CNF.

Preprocessing: Input ϕ

- replace all subformulas of the form $\psi \rightarrow \eta$ by $\neg\psi \vee \eta$ using a procedure called `IMPL_FREE`;
- formulas of the form $\neg(p \wedge q)$ may appear, we translate those to equivalent formulas that contains only negations of atoms (*negation normal form*, NNF) using NNF procedure.

2. Conjunctive normal forms and validity

CNF algorithm

Algorithm CNF:

- (1) CNF terminates for all formulas of propositional logic as input;
- (2) for each such input, CNF outputs an equivalent formula; and
- (3) all output computed by CNF is in CNF.

Preprocessing: Input ϕ

- replace all subformulas of the form $\psi \rightarrow \eta$ by $\neg\psi \vee \eta$ using a procedure called `IMPL_FREE`;
- formulas of the form $\neg(p \wedge q)$ may appear, we translate those to equivalent formulas that contains only negations of atoms (*negation normal form*, NNF) using NNF procedure.

Output $\text{NNF}(\text{IMPL_FREE})(\phi)$ contains no $\rightarrow$ and negations of only atoms.

CNF algorithm

CNF can process on only 3 cases:

- If ϕ is a literal, it is by definition in CNF and so CNF outputs ϕ .
- If ϕ equals $\phi_1 \wedge \phi_2$, we call CNF recursively on each ϕ_i to get the respective output η_i and return the CNF $\eta_1 \wedge \eta_2$ as output for input ϕ .
- If ϕ equals $\phi_1 \vee \phi_2$, we again call CNF recursively on each ϕ_i to get the respective output η_i ; but this time we must not simply return $\eta_1 \vee \eta_2$ since that formula is certainly *not* in CNF, unless η_1 and η_2 happen to be literals.

CNF algorithm

CNF can process on only 3 cases:

- If ϕ is a literal, it is by definition in CNF and so CNF outputs ϕ .
- If ϕ equals $\phi_1 \wedge \phi_2$, we call CNF recursively on each ϕ_i to get the respective output η_i and return the CNF $\eta_1 \wedge \eta_2$ as output for input ϕ .
- If ϕ equals $\phi_1 \vee \phi_2$, we again call CNF recursively on each ϕ_i to get the respective output η_i ; but this time we must not simply return $\eta_1 \vee \eta_2$ since that formula is certainly *not* in CNF, unless η_1 and η_2 happen to be literals.

Last case can be done by applying distributive law

$$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$$

using a procedure called DISTR.

CNF algorithm

function CNF (ϕ):

/* precondition: ϕ implication free and in NNF */

/* postcondition: CNF (ϕ) computes an equivalent CNF for ϕ */

begin function

case

ϕ is a literal: **return** ϕ

ϕ is $\phi_1 \wedge \phi_2$: **return** CNF (ϕ_1) $\wedge$ CNF (ϕ_2)

ϕ is $\phi_1 \vee \phi_2$: **return** DISTR (CNF (ϕ_1), CNF (ϕ_2))

end case

end function

DISTR algorithm

Suppose that $\phi = \eta_1 \vee \eta_2$

- if η_i contains no conjunctions then it contains only literals;

DISTR algorithm

Suppose that $\phi = \eta_1 \vee \eta_2$

- if η_i contains no conjunctions then it contains only literals;
- if η_i contains conjunctions $\eta_1 = \eta_{11} \wedge \eta_{12}$ then apply distributive law,

$$(\eta_{11} \wedge \eta_{12}) \vee \eta_2 = (\eta_{11} \vee \eta_2) \wedge (\eta_{12} \vee \eta_2)$$

IMPL_FREE algorithm

function IMPL_FREE(ϕ) :

/* computes a formula without implication equivalent to ϕ */

begin function

case

ϕ is an atom: **return** ϕ

ϕ is $\neg\phi_1$: **return** $\neg$ IMPL_FREE(ϕ_1)

ϕ is $\phi_1 \wedge \phi_2$: **return** IMPL_FREE(ϕ_1) $\wedge$ IMPL_FREE(ϕ_2)

ϕ is $\phi_1 \vee \phi_2$: **return** IMPL_FREE(ϕ_1) $\vee$ IMPL_FREE(ϕ_2)

ϕ is $\phi_1 \rightarrow \phi_2$: **return** IMPL_FREE($\neg\phi_1 \vee \phi_2$)

end case

end function

DISTR algorithm

function DISTR(η_1, η_2):

/* precondition: η_1 and η_2 are in CNF */

/* postcondition: DISTR(η_1, η_2) computes a CNF for $\eta_1 \vee \eta_2$ */

begin function

case

η_1 is $\eta_{11} \wedge \eta_{12}$: **return** DISTR(η_{11}, η_2) $\wedge$ DISTR(η_{12}, η_2)

η_2 is $\eta_{21} \wedge \eta_{22}$: **return** DISTR(η_1, η_{21}) $\wedge$ DISTR(η_1, η_{22})

otherwise (= no conjunctions): **return** $\eta_1 \vee \eta_2$

end case

end function

NNF algorithm

- Examples of formulas that is not in NNF: $\neg\neg p, \neg(p \wedge q), \dots$

NNF algorithm

- Examples of formulas that is not in NNF: $\neg\neg p, \neg(p \wedge q), \dots$
- double negation rule

$$\neg\neg p \equiv p$$

NNF algorithm

- Examples of formulas that is not in NNF: $\neg\neg p, \neg(p \wedge q), \dots$
- double negation rule

$$\neg\neg p \equiv p$$

- de Morgan rules:

$$\neg(\phi_1 \wedge \phi_2) \equiv \neg\phi_1 \vee \neg\phi_2$$

$$\neg(\phi_1 \vee \phi_2) \equiv \neg\phi_1 \wedge \neg\phi_2$$

NNF algorithm

```

function NNF ( $\phi$ ):
  /* precondition:  $\phi$  is implication free */
  /* postcondition: NNF ( $\phi$ ) computes a NNF for  $\phi$  */
  begin function
    case
       $\phi$  is a literal: return  $\phi$ 
       $\phi$  is  $\neg\neg\phi_1$ : return NNF ( $\phi_1$ )
       $\phi$  is  $\phi_1 \wedge \phi_2$ : return NNF ( $\phi_1$ )  $\wedge$  NNF ( $\phi_2$ )
       $\phi$  is  $\phi_1 \vee \phi_2$ : return NNF ( $\phi_1$ )  $\vee$  NNF ( $\phi_2$ )
       $\phi$  is  $\neg(\phi_1 \wedge \phi_2)$ : return NNF ( $\neg\phi_1$ )  $\vee$  NNF ( $\neg\phi_2$ )
       $\phi$  is  $\neg(\phi_1 \vee \phi_2)$ : return NNF ( $\neg\phi_1$ )  $\wedge$  NNF ( $\neg\phi_2$ )
    end case
  end function

```

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\text{IMPL_FREE}(\phi) = \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q))$$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\begin{aligned}\text{IMPL_FREE}(\phi) &= \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\ &= \neg((\text{IMPL_FREE } \neg p) \wedge (\text{IMPL_FREE } q)) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q))\end{aligned}$$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\begin{aligned}\text{IMPL_FREE}(\phi) &= \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\ &= \neg((\text{IMPL_FREE} \neg p) \wedge (\text{IMPL_FREE} q)) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\ &= \neg((\neg p) \wedge \text{IMPL_FREE} q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q))\end{aligned}$$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\begin{aligned}
 \text{IMPL_FREE}(\phi) &= \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\text{IMPL_FREE} \neg p) \wedge (\text{IMPL_FREE} q)) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\neg p) \wedge \text{IMPL_FREE} q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q))
 \end{aligned}$$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\begin{aligned}
 \text{IMPL_FREE}(\phi) &= \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\text{IMPL_FREE} \neg p) \wedge (\text{IMPL_FREE} q)) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\neg p) \wedge \text{IMPL_FREE} q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee ((\text{IMPL_FREE} p) \wedge \text{IMPL_FREE}(r \rightarrow q))
 \end{aligned}$$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\begin{aligned}
 \text{IMPL_FREE}(\phi) &= \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\text{IMPL_FREE } \neg p) \wedge (\text{IMPL_FREE } q)) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\neg p) \wedge \text{IMPL_FREE } q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee ((\text{IMPL_FREE } p) \wedge \text{IMPL_FREE}(r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge \text{IMPL_FREE}(r \rightarrow q))
 \end{aligned}$$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\begin{aligned}
 \text{IMPL_FREE}(\phi) &= \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\text{IMPL_FREE } \neg p) \wedge (\text{IMPL_FREE } q)) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\neg p) \wedge \text{IMPL_FREE } q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee ((\text{IMPL_FREE } p) \wedge \text{IMPL_FREE}(r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge \text{IMPL_FREE}(r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge (\neg(\text{IMPL_FREE } r) \vee (\text{IMPL_FREE } q)))
 \end{aligned}$$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\begin{aligned}
 \text{IMPL_FREE}(\phi) &= \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\text{IMPL_FREE } \neg p) \wedge (\text{IMPL_FREE } q)) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\neg p) \wedge \text{IMPL_FREE } q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee ((\text{IMPL_FREE } p) \wedge \text{IMPL_FREE}(r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge \text{IMPL_FREE}(r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge (\neg(\text{IMPL_FREE } r) \vee (\text{IMPL_FREE } q))) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge (\neg r \vee (\text{IMPL_FREE } q)))
 \end{aligned}$$

Example

Example 3.1

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(\neg p \wedge q \rightarrow p \wedge (r \rightarrow q))))$

$$\begin{aligned}
 \text{IMPL_FREE}(\phi) &= \neg \text{IMPL_FREE}(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\text{IMPL_FREE } \neg p) \wedge (\text{IMPL_FREE } q)) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg((\neg p) \wedge \text{IMPL_FREE } q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee \text{IMPL_FREE}(p \wedge (r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee ((\text{IMPL_FREE } p) \wedge \text{IMPL_FREE}(r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge \text{IMPL_FREE}(r \rightarrow q)) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge (\neg(\text{IMPL_FREE } r) \vee (\text{IMPL_FREE } q))) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge (\neg r \vee (\text{IMPL_FREE } q))) \\
 &= \neg(\neg p \wedge q) \vee (p \wedge (\neg r \vee q)).
 \end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\text{NNF}(\text{IMPL_FREE } \phi) = \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q))$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}\text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q))\end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}\text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (\text{NNF}(\neg\neg p)) \vee (\text{NNF}(\neg q)) \vee \text{NNF}(p \wedge (\neg r \vee q))\end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}\text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (\text{NNF}(\neg\neg p)) \vee (\text{NNF}(\neg q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee (\text{NNF}(\neg q))) \vee \text{NNF}(p \wedge (\neg r \vee q))\end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}\text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (\text{NNF}(\neg\neg p)) \vee (\text{NNF}(\neg q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee (\text{NNF}(\neg q))) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q))\end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}\text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (\text{NNF}(\neg\neg p)) \vee (\text{NNF}(\neg q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee (\text{NNF}(\neg q))) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee \neg q) \vee ((\text{NNF } p) \wedge (\text{NNF}(\neg r \vee q)))\end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}\text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (\text{NNF}(\neg\neg p)) \vee (\text{NNF}(\neg q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee (\text{NNF}(\neg q))) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\ &= (p \vee \neg q) \vee ((\text{NNF } p) \wedge (\text{NNF}(\neg r \vee q))) \\ &= (p \vee \neg q) \vee (p \wedge (\text{NNF}(\neg r \vee q)))\end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}
 \text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (\text{NNF}(\neg\neg p)) \vee (\text{NNF}(\neg q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee (\text{NNF}(\neg q))) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee \neg q) \vee ((\text{NNF } p) \wedge (\text{NNF}(\neg r \vee q))) \\
 &= (p \vee \neg q) \vee (p \wedge (\text{NNF}(\neg r \vee q))) \\
 &= (p \vee \neg q) \vee (p \wedge ((\text{NNF}(\neg r)) \vee (\text{NNF } q)))
 \end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}
 \text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (\text{NNF}(\neg\neg p)) \vee (\text{NNF}(\neg q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee (\text{NNF}(\neg q))) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee \neg q) \vee ((\text{NNF } p) \wedge (\text{NNF}(\neg r \vee q))) \\
 &= (p \vee \neg q) \vee (p \wedge (\text{NNF}(\neg r \vee q))) \\
 &= (p \vee \neg q) \vee (p \wedge ((\text{NNF}(\neg r)) \vee (\text{NNF } q))) \\
 &= (p \vee \neg q) \vee (p \wedge (\neg r \vee (\text{NNF } q)))
 \end{aligned}$$

Second, we compute $\text{NNF}(\text{IMPL_FREE } \phi)$:

$$\begin{aligned}
 \text{NNF}(\text{IMPL_FREE } \phi) &= \text{NNF}(\neg(\neg p \wedge q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= \text{NNF}(\neg(\neg p) \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (\text{NNF}(\neg\neg p)) \vee (\text{NNF}(\neg q)) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee (\text{NNF}(\neg q))) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee \neg q) \vee \text{NNF}(p \wedge (\neg r \vee q)) \\
 &= (p \vee \neg q) \vee ((\text{NNF } p) \wedge (\text{NNF}(\neg r \vee q))) \\
 &= (p \vee \neg q) \vee (p \wedge (\text{NNF}(\neg r \vee q))) \\
 &= (p \vee \neg q) \vee (p \wedge ((\text{NNF}(\neg r)) \vee (\text{NNF } q))) \\
 &= (p \vee \neg q) \vee (p \wedge (\neg r \vee (\text{NNF } q))) \\
 &= (p \vee \neg q) \vee (p \wedge (\neg r \vee q)).
 \end{aligned}$$

Third, we finish it off with

$$\text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) = \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q)))$$

Third, we finish it off with

$$\begin{aligned}\text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q)))\end{aligned}$$

Third, we finish it off with

$$\begin{aligned}\text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, \text{CNF}(p \wedge (\neg r \vee q)))\end{aligned}$$

Third, we finish it off with

$$\begin{aligned}\text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, p \wedge (\neg r \vee q))\end{aligned}$$

Third, we finish it off with

$$\begin{aligned}\text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, p \wedge (\neg r \vee q)) \\ &= \text{DISTR}(p \vee \neg q, p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q)\end{aligned}$$

Third, we finish it off with

$$\begin{aligned}\text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, p \wedge (\neg r \vee q)) \\ &= \text{DISTR}(p \vee \neg q, p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\ &= (p \vee \neg q \vee p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q)\end{aligned}$$

Third, we finish it off with

$$\begin{aligned}\text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, \text{CNF}(p \wedge (\neg r \vee q))) \\ &= \text{DISTR}(p \vee \neg q, p \wedge (\neg r \vee q)) \\ &= \text{DISTR}(p \vee \neg q, p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\ &= (p \vee \neg q \vee p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\ &= (p \vee \neg q \vee p) \wedge (p \vee \neg q \vee \neg r \vee q).\end{aligned}$$

Third, we finish it off with

$$\begin{aligned}
 \text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(p \vee \neg q, \text{CNF}(p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(p \vee \neg q, p \wedge (\neg r \vee q)) \\
 &= \text{DISTR}(p \vee \neg q, p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\
 &= (p \vee \neg q \vee p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\
 &= (p \vee \neg q \vee p) \wedge (p \vee \neg q \vee \neg r \vee q).
 \end{aligned}$$

Thus ϕ is equivalent to $(p \vee \neg q \vee p) \wedge (p \vee \neg q \vee \neg r \vee q)$

Third, we finish it off with

$$\begin{aligned}
 \text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(p \vee \neg q, \text{CNF}(p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(p \vee \neg q, p \wedge (\neg r \vee q)) \\
 &= \text{DISTR}(p \vee \neg q, p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\
 &= (p \vee \neg q \vee p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\
 &= (p \vee \neg q \vee p) \wedge (p \vee \neg q \vee \neg r \vee q).
 \end{aligned}$$

Thus ϕ is equivalent to $(p \vee \neg q \vee p) \wedge (p \vee \neg q \vee \neg r \vee q)$ then is satisfiable but not valid.

Third, we finish it off with

$$\begin{aligned}
 \text{CNF}(\text{NNF}(\text{IMPL_FREE}\phi)) &= \text{CNF}((p \vee \neg q) \vee (p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(\text{CNF}(p \vee \neg q), \text{CNF}(p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(p \vee \neg q, \text{CNF}(p \wedge (\neg r \vee q))) \\
 &= \text{DISTR}(p \vee \neg q, p \wedge (\neg r \vee q)) \\
 &= \text{DISTR}(p \vee \neg q, p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\
 &= (p \vee \neg q \vee p) \wedge \text{DISTR}(p \vee \neg q, \neg r \vee q) \\
 &= (p \vee \neg q \vee p) \wedge (p \vee \neg q \vee \neg r \vee q).
 \end{aligned}$$

Thus ϕ is equivalent to $(p \vee \neg q \vee p) \wedge (p \vee \neg q \vee \neg r \vee q)$ then is satisfiable but not valid. It is also equivalent to the simpler conjunctive normal form $p \vee \neg q$.

Example 3.2

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(r \rightarrow (s \rightarrow (t \wedge s \rightarrow r)))))$

Example 3.2

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(r \rightarrow (s \rightarrow (t \wedge s \rightarrow r))))))$

Result $\phi \equiv \neg r \vee (\neg s \vee ((\neg t \vee \neg s) \vee r))$

Example 3.2

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(r \rightarrow (s \rightarrow (t \wedge s \rightarrow r))))))$

Result $\phi \equiv \neg r \vee (\neg s \vee ((\neg t \vee \neg s) \vee r))$ then is valid.

Example 3.2

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(r \rightarrow (s \rightarrow (t \wedge s \rightarrow r))))))$

Result $\phi \equiv \neg r \vee (\neg s \vee ((\neg t \vee \neg s) \vee r))$ then is valid.

$$\begin{aligned}
 \text{IMPL_FREE}(\phi) &= \neg(\text{IMPL_FREE } r) \vee \text{IMPL_FREE}(s \rightarrow (t \wedge s \rightarrow r)) \\
 &= \neg r \vee \text{IMPL_FREE}(s \rightarrow (t \wedge s \rightarrow r)) \\
 &= \neg r \vee (\neg(\text{IMPL_FREE } s) \vee \text{IMPL_FREE}(t \wedge s \rightarrow r)) \\
 &= \neg r \vee (\neg s \vee \text{IMPL_FREE}(t \wedge s \rightarrow r)) \\
 &= \neg r \vee (\neg s \vee (\neg(\text{IMPL_FREE}(t \wedge s)) \vee \text{IMPL_FREE } r)) \\
 &= \neg r \vee (\neg s \vee (\neg((\text{IMPL_FREE } t) \wedge (\text{IMPL_FREE } s)) \vee \text{IMPL_FREE } r)) \\
 &= \neg r \vee (\neg s \vee (\neg(t \wedge (\text{IMPL_FREE } s)) \vee (\text{IMPL_FREE } r))) \\
 &= \neg r \vee (\neg s \vee (\neg(t \wedge s)) \vee (\text{IMPL_FREE } r)) \\
 &= \neg r \vee (\neg s \vee (\neg(t \wedge s)) \vee r)
 \end{aligned}$$

Example 3.2

Compute $\text{CNF}(\text{NNF}(\text{IMPL_FREE}(r \rightarrow (s \rightarrow (t \wedge s \rightarrow r))))))$

Result $\phi \equiv \neg r \vee (\neg s \vee ((\neg t \vee \neg s) \vee r))$ then is valid.

$$\begin{aligned}
 \text{IMPL_FREE}(\phi) &= \neg(\text{IMPL_FREE } r) \vee \text{IMPL_FREE}(s \rightarrow (t \wedge s \rightarrow r)) \\
 &= \neg r \vee \text{IMPL_FREE}(s \rightarrow (t \wedge s \rightarrow r)) \\
 &= \neg r \vee (\neg(\text{IMPL_FREE } s) \vee \text{IMPL_FREE}(t \wedge s \rightarrow r)) \\
 &= \neg r \vee (\neg s \vee \text{IMPL_FREE}(t \wedge s \rightarrow r)) \\
 &= \neg r \vee (\neg s \vee (\neg(\text{IMPL_FREE}(t \wedge s)) \vee \text{IMPL_FREE } r)) \\
 &= \neg r \vee (\neg s \vee (\neg((\text{IMPL_FREE } t) \wedge (\text{IMPL_FREE } s)) \vee \text{IMPL_FREE } r)) \\
 &= \neg r \vee (\neg s \vee (\neg(t \wedge (\text{IMPL_FREE } s)) \vee (\text{IMPL_FREE } r))) \\
 &= \neg r \vee (\neg s \vee (\neg(t \wedge s)) \vee (\text{IMPL_FREE } r)) \\
 &= \neg r \vee (\neg s \vee (\neg(t \wedge s)) \vee r)
 \end{aligned}$$

$$\begin{aligned}
\text{NNF } (\text{IMPL_FREE } \phi) &= \text{NNF } (\neg r \vee (\neg s \vee \neg(t \wedge s) \vee r)) \\
&= (\text{NNF } \neg r) \vee \text{NNF } (\neg s \vee \neg(t \wedge s) \vee r) \\
&= \neg r \vee \text{NNF } (\neg s \vee \neg(t \wedge s) \vee r) \\
&= \neg r \vee (\text{NNF } (\neg s) \vee \text{NNF } (\neg(t \wedge s) \vee r)) \\
&= \neg r \vee (\neg s \vee \text{NNF } (\neg(t \wedge s) \vee r)) \\
&= \neg r \vee (\neg s \vee (\text{NNF } (\neg(t \wedge s)) \vee \text{NNF } r)) \\
&= \neg r \vee (\neg s \vee (\text{NNF } (\neg t \vee \neg s)) \vee \text{NNF } r) \\
&= \neg r \vee (\neg s \vee ((\text{NNF } (\neg t) \vee \text{NNF } (\neg s)) \vee \text{NNF } r)) \\
&= \neg r \vee (\neg s \vee ((\neg t \vee \text{NNF } (\neg s)) \vee \text{NNF } r)) \\
&= \neg r \vee (\neg s \vee ((\neg t \vee \neg s) \vee \text{NNF } r)) \\
&= \neg r \vee (\neg s \vee ((\neg t \vee \neg s) \vee r))
\end{aligned}$$

$$\begin{aligned}
\text{NNF } (\text{IMPL_FREE } \phi) &= \text{NNF } (\neg r \vee (\neg s \vee \neg(t \wedge s) \vee r)) \\
&= (\text{NNF } \neg r) \vee \text{NNF } (\neg s \vee \neg(t \wedge s) \vee r) \\
&= \neg r \vee \text{NNF } (\neg s \vee \neg(t \wedge s) \vee r) \\
&= \neg r \vee (\text{NNF } (\neg s) \vee \text{NNF } (\neg(t \wedge s) \vee r)) \\
&= \neg r \vee (\neg s \vee \text{NNF } (\neg(t \wedge s) \vee r)) \\
&= \neg r \vee (\neg s \vee (\text{NNF } (\neg(t \wedge s)) \vee \text{NNF } r)) \\
&= \neg r \vee (\neg s \vee (\text{NNF } (\neg t \vee \neg s)) \vee \text{NNF } r) \\
&= \neg r \vee (\neg s \vee ((\text{NNF } (\neg t) \vee \text{NNF } (\neg s)) \vee \text{NNF } r)) \\
&= \neg r \vee (\neg s \vee ((\neg t \vee \text{NNF } (\neg s)) \vee \text{NNF } r)) \\
&= \neg r \vee (\neg s \vee ((\neg t \vee \neg s) \vee \text{NNF } r)) \\
&= \neg r \vee (\neg s \vee ((\neg t \vee \neg s) \vee r))
\end{aligned}$$

3. Horn clauses and satisfiability

- CNF has an easy syntactic check for validity but its test for satisfiability is very hard in general.

3. Horn clauses and satisfiability

- CNF has an easy syntactic check for validity but its test for satisfiability is very hard in general.
- there are important subclasses of formulas which have much more efficient ways of deciding their satisfiability. One of them is the class of *Horn formulas*.

3. Horn clauses and satisfiability

- CNF has an easy syntactic check for validity but its test for satisfiability is very hard in general.
- there are important subclasses of formulas which have much more efficient ways of deciding their satisfiability. One of them is the class of *Horn formulas*.

Definition 4.1

A *Horn formula* is a formula ϕ of propositional logic if it can be generated as an instance of H in this grammar:

$$P ::= \perp \mid \top \mid p$$

$$A ::= P \mid P \wedge A$$

$$C ::= A \rightarrow P$$

$$H ::= C \mid C \wedge H.$$

We call each instance of C a *Horn clause*.

Horn formulas

$$P ::= \perp \mid \top \mid p$$

$$A ::= P \mid P \wedge A$$

$$C ::= A \rightarrow P$$

$$H ::= C \mid C \wedge H.$$

Horn formulas

$$P ::= \perp \mid \top \mid p$$

$$A ::= P \mid P \wedge A$$

$$C ::= A \rightarrow P$$

$$H ::= C \mid C \wedge H.$$

- Horn formulas are conjunctions of Horn clauses.

Horn formulas

$$P ::= \perp \mid \top \mid p$$

$$A ::= P \mid P \wedge A$$

$$C ::= A \rightarrow P$$

$$H ::= C \mid C \wedge H.$$

- Horn formulas are conjunctions of Horn clauses.
- A Horn clause is an implication whose assumption A is a conjunction of propositions of type P and whose conclusion is also of type P .

Examples

Example 4.2

The following formulas are Horn formulas

$$(p \wedge q \wedge s \rightarrow p) \wedge (q \wedge r \rightarrow p) \wedge (p \wedge s \rightarrow s)$$

$$(p \wedge q \wedge s \rightarrow \perp) \wedge (q \wedge r \rightarrow p) \wedge (\top \rightarrow s)$$

$$(p_2 \wedge p_3 \wedge p_5 \rightarrow p_{13}) \wedge (\top \rightarrow p_5) \wedge (p_5 \wedge p_{11} \rightarrow \perp).$$

Examples

Example 4.3

The following formulas are **NOT** Horn formulas

$$(p \wedge q \wedge s \rightarrow \neg p) \wedge (q \wedge r \rightarrow p) \wedge (p \wedge s \rightarrow s)$$

$$(p \wedge q \wedge s \rightarrow \perp) \wedge (\neg q \wedge r \rightarrow p) \wedge (\top \rightarrow s)$$

$$(p_2 \wedge p_3 \wedge p_5 \rightarrow p_{13} \wedge p_{27}) \wedge (\top \rightarrow p_5) \wedge (p_5 \wedge p_{11} \rightarrow \perp)$$

$$(p_2 \wedge p_3 \wedge p_5 \rightarrow p_{13} \wedge p_{27}) \wedge (\top \rightarrow p_5) \wedge (p_5 \wedge p_{11} \vee \perp).$$

Horn formulas and Satisfiability

The algorithm we propose for deciding the satisfiability of a Horn formula ϕ maintains a list of all occurrences of type P in ϕ and proceeds like this:

1. It marks $\top$ if it occurs in that list.
2. If there is a conjunct $P_1 \wedge P_2 \wedge \cdots \wedge P_{k_i} \rightarrow P'$ of ϕ such that all P_j with $1 \leq j \leq k_i$ are marked, mark P' as well and go to 2. Otherwise (= there is no conjunct $P_1 \wedge P_2 \wedge \cdots \wedge P_{k_i} \rightarrow P'$ such that all P_j are marked) go to 3.
3. If $\perp$ is marked, print out 'The Horn formula ϕ is unsatisfiable.' and stop. Otherwise, go to 4.
4. Print out 'The Horn formula ϕ is satisfiable.' and stop.

Horn formulas and Satisfiability

function HORN(ϕ):

/* precondition: ϕ is a Horn formula */

/* postcondition: HORN(ϕ) decides the satisfiability for ϕ */

begin function

mark all occurrences of $\top$ in ϕ ;

while there is a conjunct $P_1 \wedge P_2 \wedge \dots \wedge P_{k_i} \rightarrow P'$ of ϕ
such that all P_j are marked but P' isn't **do**
mark P'

end while

if $\perp$ is marked **then return** 'unsatisfiable' **else return** 'satisfiable'

end function

Example

Apply algorithm HORN to each of these Horn formulas

$$\textcircled{1} (\top \rightarrow q) \wedge (\top \rightarrow s) \wedge (w \rightarrow \perp) \wedge (p \wedge q \wedge s \rightarrow \perp) \wedge (v \rightarrow s) \wedge (\top \rightarrow r) \wedge (r \rightarrow p)$$

Example

Apply algorithm HORN to each of these Horn formulas

- ❶ $(\top \rightarrow q) \wedge (\top \rightarrow s) \wedge (w \rightarrow \perp) \wedge (p \wedge q \wedge s \rightarrow \perp) \wedge (v \rightarrow s) \wedge (\top \rightarrow r) \wedge (r \rightarrow p)$
- ❷ $(p \wedge q \wedge w \rightarrow \perp) \wedge (t \rightarrow \perp) \wedge (r \rightarrow p) \wedge (\top \rightarrow r) \wedge (\top \rightarrow q) \wedge (u \rightarrow s) \wedge (\top \rightarrow u)$

Horn formulas and Satisfiability

Theorem 4.4

The algorithm HORN is correct for the satisfiability decision problem of Horn formulas and has no more than $n + 1$ cycles in its while-statement if n is the number of atoms in ϕ . In particular, HORN always terminates on correct input.

4. SAT solver

A linear solver

We will execute marking algorithm on the parse tree of formulas, except that we will translate formulas into the adequate fragment

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi)$$

4. SAT solver

A linear solver

We will execute marking algorithm on the parse tree of formulas, except that we will translate formulas into the adequate fragment

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi)$$

then share common subformulas of the resulting parse tree, making the tree into a directed, acyclic graph (DAG).

4. SAT solver

A linear solver

We will execute marking algorithm on the parse tree of formulas, except that we will translate formulas into the adequate fragment

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi)$$

then share common subformulas of the resulting parse tree, making the tree into a directed, acyclic graph (DAG).

The inductively defined translations

$$T(p) = p$$

$$T(\phi_1 \wedge \phi_2) = T(\phi_1) \wedge T(\phi_2)$$

$$T(\phi_1 \rightarrow \phi_2) = \neg(T(\phi_1) \wedge \neg T(\phi_2))$$

$$T(\neg\phi) = \neg T(\phi)$$

$$T(\phi_1 \vee \phi_2) = \neg(\neg T(\phi_1) \wedge \neg T(\phi_2))$$

4. SAT solver

A linear solver

We will execute marking algorithm on the parse tree of formulas, except that we will translate formulas into the adequate fragment

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi)$$

then share common subformulas of the resulting parse tree, making the tree into a directed, acyclic graph (DAG).

The inductively defined translations

$$T(p) = p$$

$$T(\neg\phi) = \neg T(\phi)$$

$$T(\phi_1 \wedge \phi_2) = T(\phi_1) \wedge T(\phi_2)$$

$$T(\phi_1 \vee \phi_2) = \neg(\neg T(\phi_1) \wedge \neg T(\phi_2))$$

$$T(\phi_1 \rightarrow \phi_2) = \neg(T(\phi_1) \wedge \neg T(\phi_2))$$

Note that ϕ and $T(\phi)$ are semantically equivalent.

4. SAT solver

A linear solver

We will execute marking algorithm on the parse tree of formulas, except that we will translate formulas into the adequate fragment

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi)$$

then share common subformulas of the resulting parse tree, making the tree into a directed, acyclic graph (DAG).

The inductively defined translations

$$T(p) = p$$

$$T(\neg\phi) = \neg T(\phi)$$

$$T(\phi_1 \wedge \phi_2) = T(\phi_1) \wedge T(\phi_2)$$

$$T(\phi_1 \vee \phi_2) = \neg(\neg T(\phi_1) \wedge \neg T(\phi_2))$$

$$T(\phi_1 \rightarrow \phi_2) = \neg(T(\phi_1) \wedge \neg T(\phi_2))$$

Note that ϕ and $T(\phi)$ are semantically equivalent.

This SAT solver has a linear running time in the size of the DAG for $T(\phi)$.

Example

Example 5.1

Let $\phi := p \wedge \neg(q \vee \neg p)$.

Example

Example 5.1

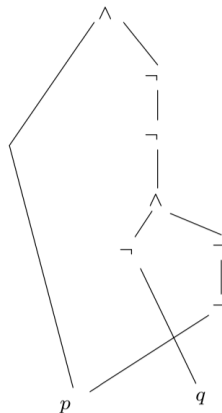
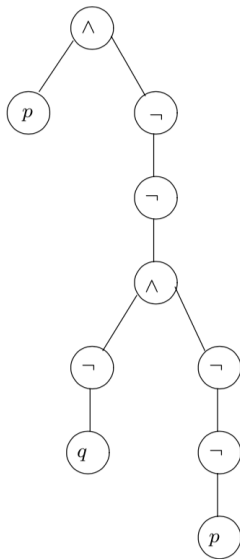
Let $\phi := p \wedge \neg(q \vee \neg p)$. We compute
 $T(\phi) = p \wedge \neg\neg(\neg q \wedge \neg\neg p)$.

Example

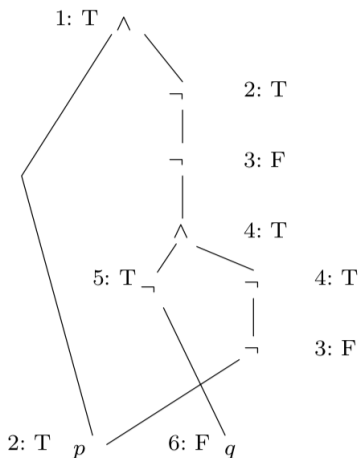
Example 5.1

Let $\phi := p \wedge \neg(q \vee \neg p)$. We compute $T(\phi) = p \wedge \neg\neg(\neg q \wedge \neg\neg p)$. Construct parse tree and DAG of $T(\phi)$.

$$T(\phi) = p \wedge \neg\neg(\neg q \wedge \neg\neg p).$$

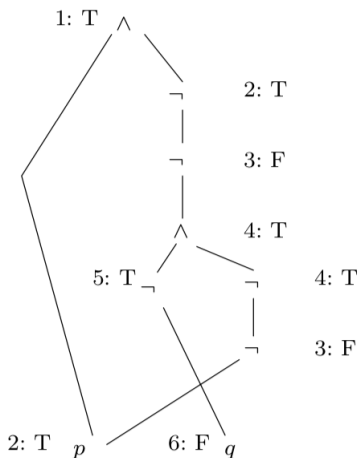


$$T(\phi) = p \wedge \neg\neg(\neg q \wedge \neg\neg p).$$



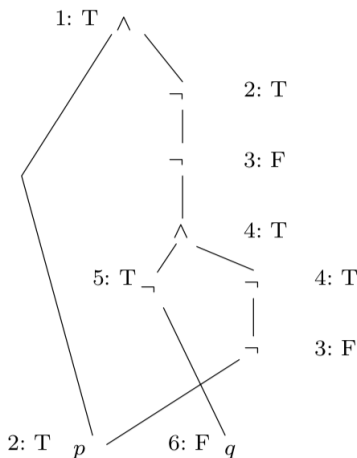
Recompute values of nodes from bottom up to see everything matches.

$$T(\phi) = p \wedge \neg\neg(\neg q \wedge \neg\neg p).$$



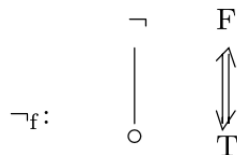
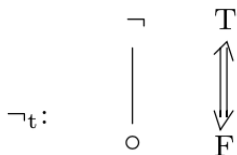
Recompute values of nodes from bottom up to see everything matches. There is a *witness* to the satisfiability of the formula represented by this DAG.

$$T(\phi) = p \wedge \neg\neg(\neg q \wedge \neg\neg p).$$

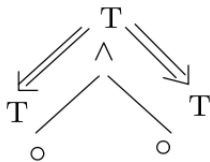
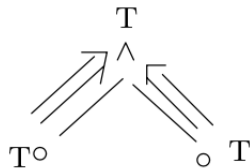


Recompute values of nodes from bottom up to see everything matches. There is a *witness* to the satisfiability of the formula represented by this DAG. **Thus** $T(\phi)$ is satisfiable and so is ϕ .

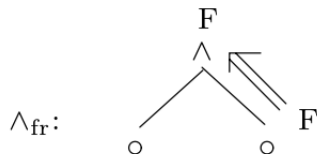
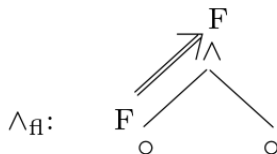
Rules for flow constraints



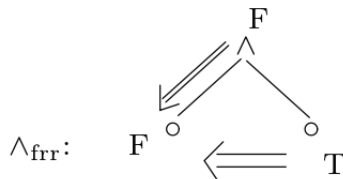
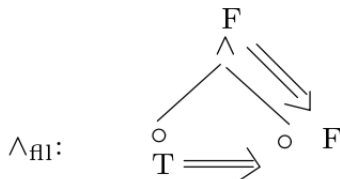
Rules for flow constraints

 $\wedge_{te}:$

 $\wedge_{ti}:$


Rules for flow constraints



Rules for flow constraints



Example

Example 5.2

To verify the sequent $p \wedge q \rightarrow r \vdash p \rightarrow q \rightarrow r$ valid,

Example

Example 5.2

To verify the sequent $p \wedge q \rightarrow r \vdash p \rightarrow q \rightarrow r$ valid, we check the validity of $(p \wedge q \rightarrow r) \rightarrow p \rightarrow q \rightarrow r$,

Example

Example 5.2

To verify the sequent $p \wedge q \rightarrow r \vdash p \rightarrow q \rightarrow r$ valid, we check the validity of $(p \wedge q \rightarrow r) \rightarrow p \rightarrow q \rightarrow r$, and it is equivalent to $\phi = \neg((p \wedge q \rightarrow r) \rightarrow p \rightarrow q \rightarrow r)$ is not satisfiable.

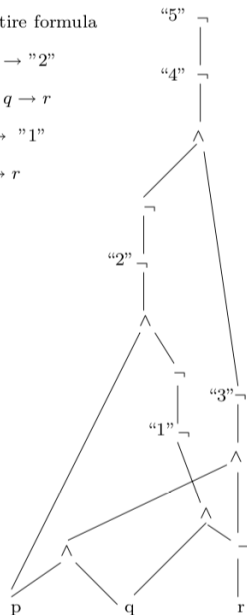
"5" = entire formula

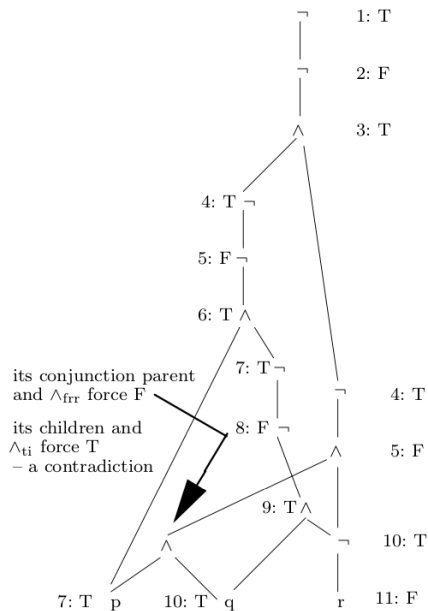
"4" = "3" $\rightarrow$ "2"

"3" = $p \wedge q \rightarrow r$

"2" = $p \rightarrow$ "1"

"1" = $q \rightarrow r$





Example

Example 5.3

Check the validity of $p \vee q, p \rightarrow r \vdash r$

Example

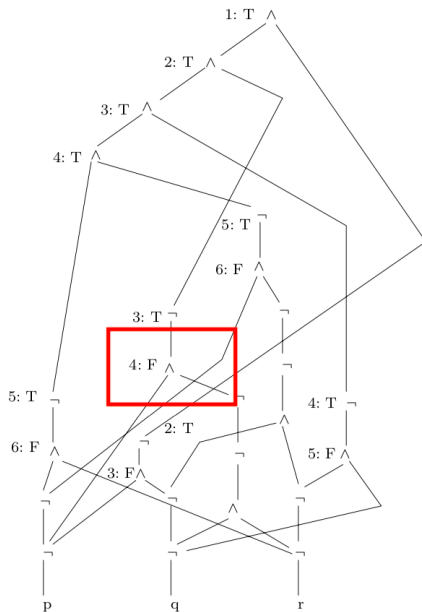
Example 5.3

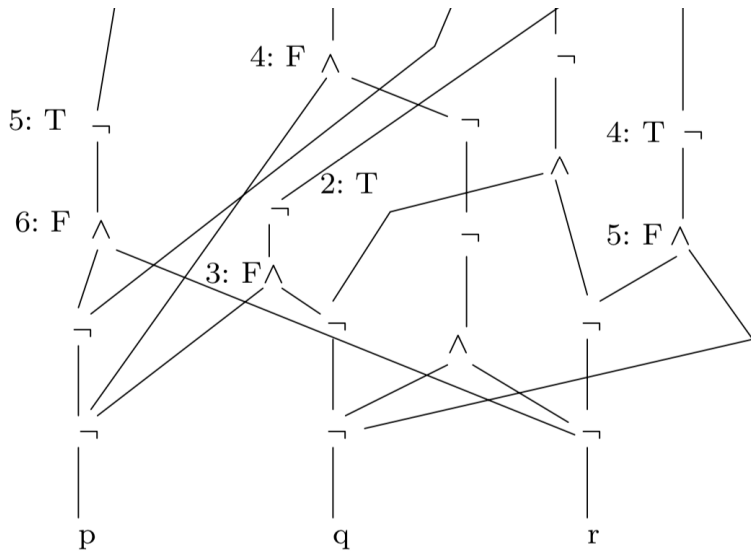
Check the validity of $p \vee q, p \rightarrow r \vdash r$

Example 5.4

Check the satisfiability of the following

$$((p \vee (q \vee r)) \wedge ((p \vee \neg q) \wedge ((q \vee \neg r) \wedge ((r \vee \neg p) \wedge (\neg p \vee (\neg q \vee \neg r)))))))$$



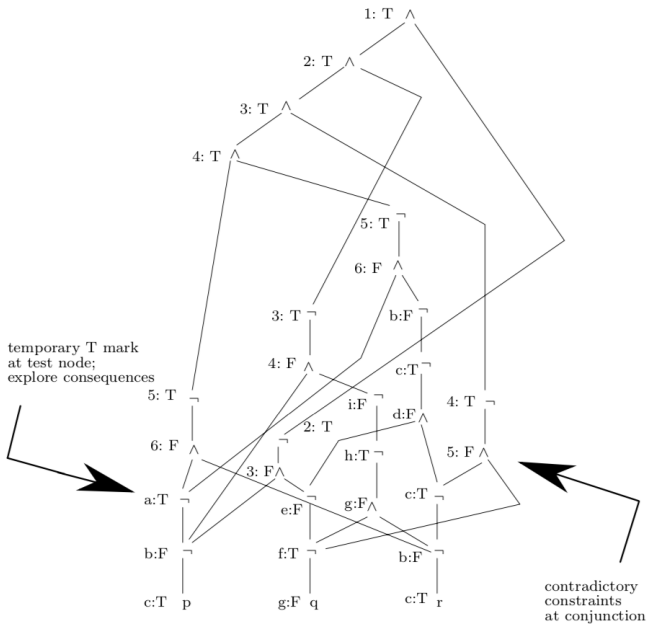


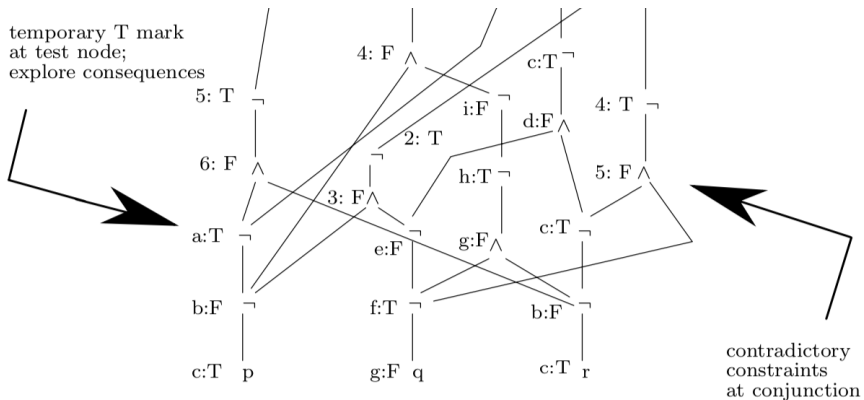
A cubic solver

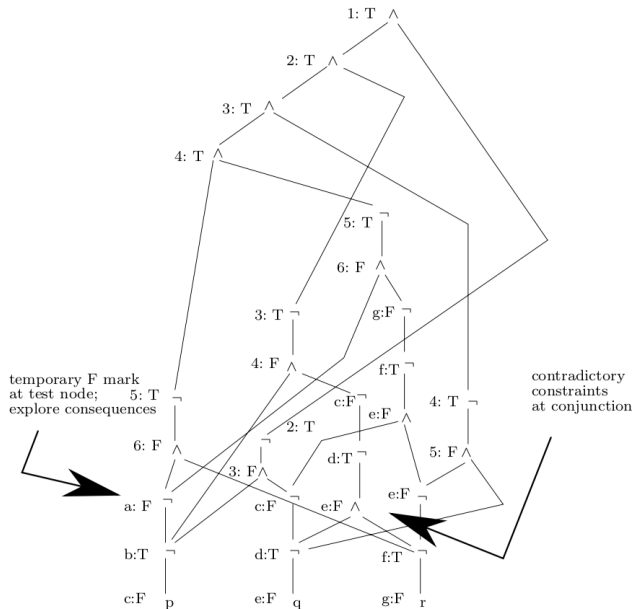
In the DAG, pick any node n that is not yet marked. Then test node n by making two independent computations:

- determine which temporary marks are forced by adding to the marks the T mark only to n ; and
- determine which temporary marks are forced by adding to the marks the F mark only to n .

If both runs find contradictory constraints, the algorithm stops and reports that $T(\phi)$ is unsatisfiable. Otherwise, all nodes that received the same mark in both of these runs receive that very mark as a permanent one; that is, we update the mark state with all such shared marks.







Example

The formula

$$((p \vee (q \vee r)) \wedge ((p \vee \neg q) \wedge ((q \vee \neg r) \wedge ((r \vee \neg p) \wedge (\neg p \vee (\neg q \vee \neg r)))))))$$

is not satisfiable.